Sajilo Khata

Nepal SMS Expense Tracker & Savings Goals

Technical Documentation • v1.0

Built with Flutter + Firebase • Supports all major Nepal banks & wallets • BS/AD dual calendar

1. Project Overview

Sajilo Khata is a Flutter mobile application that automatically tracks income and expenses by reading SMS messages from Nepali banks and digital wallets. Users can also add transactions manually. All data is stored in Firebase Firestore and syncs across devices on login. A savings goals feature lets users set targets (e.g. "Rs. 80,000 for a laptop by Dashain") and track progress over time.

1.1 Why This App

- No existing Nepali expense tracker parses bank SMS automatically
- Existing apps have poor UX and lack BS date support
- Integrates the developer's own Nepali Calendar Kit package
- Real daily utility — used for NPR, designed for Nepali spending patterns

1.2 Tech Stack

Layer	Technology
UI Framework	Flutter 3.x (Dart)
State Management	BLoC pattern (flutter_bloc)
Authentication	Firebase Auth — Google + Email/Password
Database	Cloud Firestore (real-time sync)
Local Cache	Hive (offline-first)
SMS Reading	telephony package (Android)

Charts	fl_chart
Notifications	flutter_local_notifications + FCM
Nepali Dates	nepali_calendar_kit (own package)
ID Generation	uuid

---

2. Feature List

2.1 Authentication

Feature	Description	Week
Google Sign-In	One-tap login via Google account	Week 1
Email/Password	Standard email signup and login	Week 1
Profile setup	Name, currency preference (NPR default)	Week 1
Device sync	Login on any device — data loads from Firestore	Week 1

2.2 SMS Auto-Reader

Feature	Description	Week
Auto SMS read	Reads incoming bank SMS on Android (READ_SMS permission)	Week 1
Bank parsers	Regex parsers for 7+ Nepal banks and wallets	Week 1
Data extraction	Extracts amount, debit/credit, bank name, datetime	Week 1
Manual fallback	User pastes SMS text — app parses and logs (iOS + unknown)	Week 1
Unknown SMS	Unrecognized messages are silently ignored	Week 1

2.3 Manual Transactions

Feature	Description	Week
Add expense/income	Quick-entry form: amount, category, note, date	Week 1
Edit transaction	Edit any field of SMS-parsed or manual transactions	Week 1
Delete transaction	Soft delete with confirmation dialog	Week 1

Auto-categorize	Keyword matching: Bhatbhateni → Food, Pathao → Transport	Week 1

2.4 Firebase Sync

Feature	Description	Week
Firestore storage	All transactions stored under users/{uid}/transactions	Week 1
Offline-first	Hive local cache — works without internet, syncs on reconnect	Week 1
Real-time updates	Stream-based — UI updates instantly on data change	Week 1

2.5 Dashboard & Reports

Feature	Description	Week
Monthly summary	Total income, expenses, and net balance for the month	Week 2
Pie chart	Spending breakdown by category (fl_chart)	Week 2
Bar chart	Daily spend over the last 30 days	Week 2
BS / AD toggle	Switch between Bikram Sambat and Gregorian dates	Week 2
Transaction list	Scrollable list — debit in red, credit in green	Week 2

2.6 Savings Goals

Feature	Description	Week
Create goal	Name, emoji, target amount, and deadline (BS or AD)	Week 2
Manual contribute	Add savings to any goal at any time	Week 2
Auto-contribute	On credit SMS, prompt: "Add to a goal?"	Week 2
Progress bar	Visual progress with % saved and amount remaining	Week 2
Status badge	On Track / Behind / Achieved — calculated from deadline	Week 2
Daily target	"Save Rs. X/day" — remaining ÷ days left	Week 2
Multiple goals	Unlimited goals, sorted by nearest deadline	Week 2
Goal achieved alert	Push notification when 100% is reached	Week 2

2.7 Notifications & Export

Feature	Description	Week
SMS log alert	Push notification when new transaction is auto-logged	Week 2
Budget alert	Notify at 80% and 100% of monthly budget limit	Week 2
Goal alert	Celebrate when a savings goal is achieved	Week 2
CSV export	Export all transactions as a CSV file	Week 2

---

3. Project Structure

The project follows a feature-first architecture. Each feature is self-contained with its own screens, BLoC, repository, and widgets. Core utilities and models are shared across features.

3.1 Top-Level Structure

Path	Purpose
`lib/main.dart`	App entry point — Firebase init, BLoC providers, routing
`lib/core/`	Shared code: models, services, utils, constants
`lib/features/`	One folder per feature, self-contained
`pubspec.yaml`	Package dependencies
`android/AndroidManifest.xml`	SMS permissions (READ_SMS, RECEIVE_SMS)

3.2 Core Layer

Path	Purpose
`core/models/transaction_model.dart`	TransactionModel — data class + Firestore serialization
`core/models/goal_model.dart`	GoalModel — includes progress/status calculations
`core/services/firebase_service.dart`	All Firestore reads, writes, streams — single source of truth
`core/utils/categorizer.dart`	Keyword → category auto-assignment
`core/constants/app_theme.dart`	Light & dark MaterialTheme definitions

3.3 Features Layer

Path	Purpose
`features/auth/bloc/`	AuthBloc — Google/email login, logout, session check
`features/auth/repository/`	AuthRepository — wraps Firebase Auth calls
`features/auth/screens/`	LoginScreen, SignupScreen, ProfileSetupScreen
`features/sms/parser/sms_parser.dart`	Regex parsers for all supported banks
`features/sms/service/`	SmsListenerService — listens for new SMS on Android
`features/transactions/bloc/`	TransactionBloc — add, edit, delete, list
`features/transactions/repository/`	TransactionRepository — calls FirebaseService
`features/transactions/screens/`	TransactionListScreen, AddTransactionScreen
`features/transactions/widgets/`	TransactionTile, CategoryChip, AmountBadge
`features/goals/bloc/`	GoalBloc — create, contribute, status update
`features/goals/repository/`	GoalRepository — calls FirebaseService
`features/goals/screens/`	GoalsListScreen, AddGoalScreen, GoalDetailScreen
`features/goals/widgets/`	GoalCard, ProgressBar, StatusBadge
`features/dashboard/screens/`	DashboardScreen — monthly summary + charts
`features/dashboard/widgets/`	SummaryCard, PieChartWidget, BarChartWidget

---

4. Data Models

4.1 TransactionModel

Field	Type	Notes
`id`	`String`	UUID v4 — Firestore document ID
`amount`	`double`	Transaction amount in NPR
`type`	`enum`	debit or credit
`source`	`enum`	sms or manual
`category`	`String`	Auto-assigned or user-selected
`bank`	`String?`	Bank/wallet name from SMS parser
`note`	`String?`	Optional user note or SMS description
`dateAD`	`DateTime`	Gregorian transaction date
`dateBS`	`String`	Bikram Sambat date (from calendar kit)

createdAt	DateTime	Record creation timestamp

4.2 GoalModel

Field	Type	Notes
id	String	UUID v4
name	String	Goal title e.g. "New Laptop"
emoji	String	Visual identifier e.g. 🖥️
targetAmount	double	Total savings needed in NPR
savedAmount	double	Amount saved so far
deadlineAD	DateTime	Target date in Gregorian
deadlineBS	String	Target date in Bikram Sambat
status	enum	onTrack | behind | achieved
progressPercent	double	Computed: savedAmount / targetAmount
requiredDailyAmount	double	Computed: remaining / daysLeft

---

5. Firestore Database Schema

All user data is scoped under users/{uid}/ to ensure privacy and cross-device sync. Firestore security rules should restrict read/write access to the authenticated owner only.

users/{uid}/ profile: { name, currency, updatedAt } transactions/{txId}: { amount, type, source, category, bank, note, dateAD, dateBS, createdAt } goals/{goalId}: { name, emoji, targetAmount, savedAmount, deadlineAD, deadlineBS, status, createdAt }

5.1 Firestore Security Rules

```
rules_version = '2'; service cloud.firestore {   match /databases/{database}/documents {
match /users/{userId}/{document=**} {       allow read, write: if request.auth != null
&& request.auth.uid == userId;     }   } }
```

---

6. SMS Parser

6.1 Supported Banks & Wallets

Bank / Wallet	Sample SMS Pattern	Amount Pattern
eSewa	`"Rs. 500 debited from eSewa"`	`Rs. {amount}`
Khalti	`"Payment of NPR 300 successful"`	`NPR {amount}`
NIC Asia	`"A/C XX1234 debited NPR 1,500"`	`NPR {amount}`
Nabil Bank	`"NPR 2000 has been debited"`	`NPR {amount}`
Global IME	`"a/c debited by NPR 3,200"`	`NPR {amount}`
Prabhu Bank	`"debited NPR 500 from your account"`	`NPR {amount}`
Laxmi Sunrise	`"credited NPR 10,000 to your account"`	`NPR {amount}`

6.2 How the Parser Works

- Check sender name against known bank identifiers
- Look for debit/credit keywords in the message body
- Extract NPR amount using regex: NPR {digits} or Rs. {digits}
- Return ParsedSms(amount, type, bank) or null if unrecognized
- Unknown/promotional SMS are silently ignored — no false entries

---

7. Android Setup — SMS Permissions

Add the following to android/app/src/main/AndroidManifest.xml inside the <manifest> tag:

```
<uses-permission android:name="android.permission.READ_SMS"/> <uses-permission
android:name="android.permission.RECEIVE_SMS"/>
```

Request permissions at runtime in Flutter:

```
import 'package:permission_handler/permission_handler.dart';  future<void>
requestSmsPermission() async {   await Permission.sms.request(); }
```

> Important: Google Play Store requires a Privacy Policy URL and a declaration explaining why READ_SMS is needed. Prepare this before publishing. The review typically takes 3–7 business days.

---

8. Auto-Category System

The Categorizer utility in core/utils/categorizer.dart maps keywords in transaction notes or bank names to categories. This list should be expanded over time based on real user data.

Category	Sample Keywords
Food & Dining	bhatbhateni, foodmandu, cafe, pizza, momo, restaurant
Transport	pathao, indrive, tootle, taxi, bus, petrol, fuel
Shopping	daraz, sastodeal, bigmart, fashion, mall
Utilities	nea, ntc, ncell, broadband, dish home
Health	hospital, clinic, pharmacy, medical
Education	school, college, tuition, course, exam fee
Remittance	imepay, western union, money transfer, sent to
Salary / Income	salary, payroll, wages, bonus

---

9. Build Timeline (2 Weeks)

Week 1 — Core

- Day 1–2: Firebase setup, Auth (Google + Email), Firestore rules
- Day 3–4: TransactionModel, FirebaseService, Hive offline cache
- Day 5–6: SMS parser for all 7 banks, SmsListenerService
- Day 7: Manual add/edit/delete transaction screens

Week 2 — Polish

- Day 8–9: Dashboard — monthly summary, BS/AD toggle
- Day 10: Charts — pie chart by category, bar chart daily spend
- Day 11–12: Savings goals — create, contribute, progress bar
- Day 13: Push notifications — transaction logged, goal achieved
- Day 14: CSV export, testing, bug fixes, README

---

10. Portfolio Talking Points

When presenting this project to hirers, highlight:

- SMS parsing pipeline — regex, multi-bank support, graceful fallback
- Offline-first architecture — Hive cache + Firestore sync

- BLoC state management — scalable, testable, production patterns
- Cross-platform considerations — Android SMS vs iOS manual
- Real-world utility — solves a genuine pain point for Nepali users
- Nepali Calendar Kit integration — your own published package
- Savings goal smart calculation — daily savings target algorithm

This project demonstrates: advanced Flutter patterns, Firebase integration, background services, data parsing, chart rendering, and local/remote sync — all in a single coherent product targeting a real market gap.

Built by Gambhir Poudel • gambhirpoudel.com.np